

PKS_B Computer and Communication Networks

Communication application using the UDP protocol

Kosharova Olena 128372

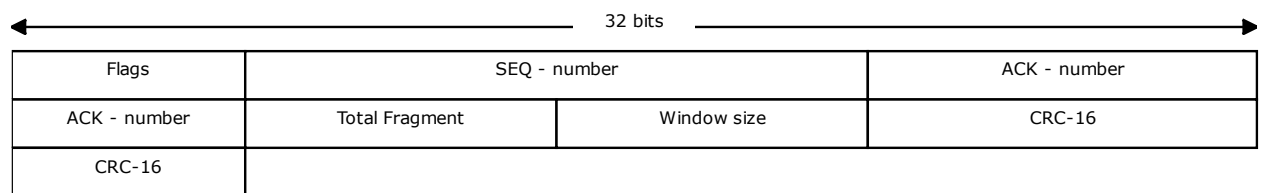
Content:

The goal of the assignment.....	3
The header structure of the protocol.....	3
The structure of the program	4
Verify the basic functionality of the connection	5
Fragmentation	6
Integrity verification	7
Automatic Repeat Request (ARQ)	8
Maintain the connection (Keep-Alive)	9
Connection completion	10
Make a mistake intentionally	12
Conclusion	12

The goal of the assignment

To design and implement a P2P application using a custom protocol built on top of User Datagram Protocol (UDP) in the transport layer of the TCP/IP network model. The application will allow communication between two participants in a local Ethernet network, including text exchange and transfer of files between computers (nodes). The two nodes will act simultaneously as the receiver and sender.

The header structure of the protocol



This header consists of 80 bits.

Flags (7 bits):

- SYN – synchronize, needed to establish a connection
- ACK – acknowledgment, needed to establish a connection
- NACK – negative acknowledgment, needed to indicate that a received packet was not correctly received and needs to be retransmitted.
- KA – keep-alive, needed to maintain a connection
- RST – reset connection, needed for ending connection without acknowledgment of another side
- FIN – finish connection, needed for ending connection without acknowledgment of another side
- DATA:
 - 0 – text sent
 - 1 – file sent

SEQ – number (16 bits): sequence number that indicates the order of a packet in the data stream. This is important for the correct assembly of the data received.

ACK – number (16 bits): acknowledgment number of the packet received by the sender. Used for confirmation of successful delivery of fragments.

Total fragment (16 bits): indicates the total number of data fragments that must be transferred.

CRC-16: data integrity check field. Calculated using the CRC-16 algorithm to detect errors in the transmitted data.

The Nack flag was added to the header for proper ARQ processing.

The structure of the program

[illegible]

The structure of the Keep-Alive changed due to implementation with thread. The structure of the ARQ became more visible. There were added new functions to listen because of the added NACK flag.

Verify the basic functionality of the connection

3-Way Handshake

In a 3-way handshake, three steps occur between two nodes before they start transmitting data.

Step 1: SYN (Synchronize)

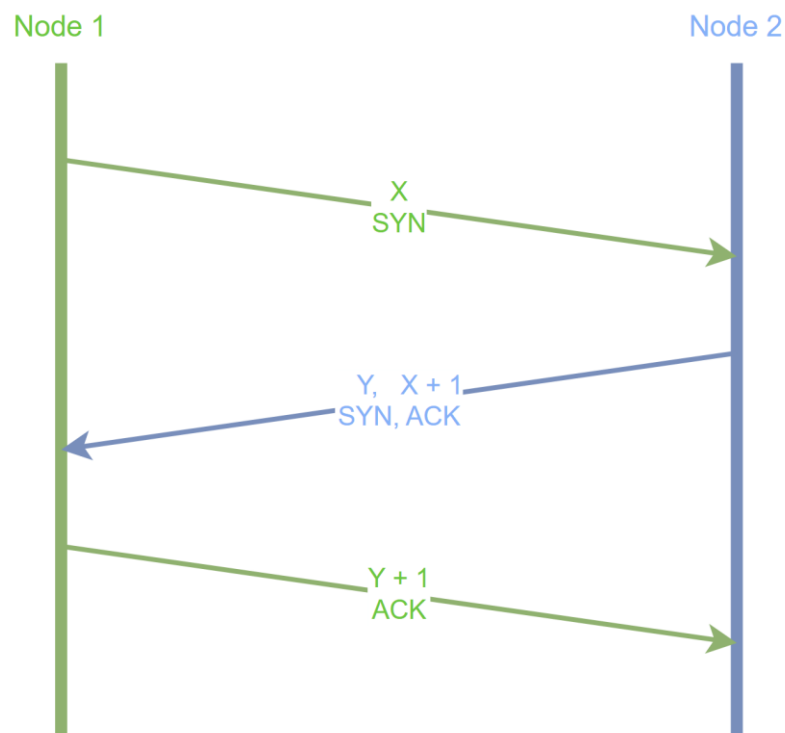
The first node initiates the handshake by sending a SYN packet to the second node. This packet contains an initial sequence number, and it indicates the first node's desire to start a communication session

Step 2: SYN-ACK (Synchronize + Acknowledge)

The server responds with a SYN-ACK packet. This response acknowledges the client's SYN packet (it sends back $X + 1$) and includes its own sequence number, signaling that the second node is ready to establish a connection.

Step 3: ACK (Acknowledge)

The first node completes the handshake by sending an ACK packet. This acknowledges the second node's sequence number ($Y + 1$). At this point, both nodes have agreed on each other's sequence numbers, and the connection is considered established.



86769	4757.325353	127.0.0.1	127.0.0.1	PKS	42 SYN
86770	4757.325966	127.0.0.1	127.0.0.1	PKS	42 SYN-ACK
86771	4757.326579	127.0.0.1	127.0.0.1	PKS	42 ACK

86769	4757.325353	127.0.0.1	127.0.0.1	PKS	42 SYN
86770	4757.325966	127.0.0.1	127.0.0.1	PKS	42 SYN-ACK
86771	4757.326579	127.0.0.1	127.0.0.1	PKS	42 ACK

Frame 86769: 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface Null/Loopback

Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1

User Datagram Protocol, Src Port: 2222, Dst Port: 1111

Custom Protocol: Control Message

Flags: 0x80

Sequence Number: 676

Acknowledgment Number: 0

Total Fragments: 1

Window Size: 0

Checksum: 0x0000

86769	4757.325353	127.0.0.1	127.0.0.1	PKS	42 SYN
86770	4757.325966	127.0.0.1	127.0.0.1	PKS	42 SYN-ACK
86771	4757.326579	127.0.0.1	127.0.0.1	PKS	42 ACK

Frame 86770: 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface Null/Loopback

Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1

User Datagram Protocol, Src Port: 1111, Dst Port: 2222

Custom Protocol: Control Message

Flags: 0xc0

Sequence Number: 957

Acknowledgment Number: 677

Total Fragments: 1

Window Size: 0

Checksum: 0x0000

86769	4757.325353	127.0.0.1	127.0.0.1	PKS	42 SYN
86770	4757.325966	127.0.0.1	127.0.0.1	PKS	42 SYN-ACK
86771	4757.326579	127.0.0.1	127.0.0.1	PKS	42 ACK

Frame 86771: 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface Null/Loopback

Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1

User Datagram Protocol, Src Port: 2222, Dst Port: 1111

Custom Protocol: Control Message

Flags: 0x40

Sequence Number: 677

Acknowledgment Number: 958

Total Fragments: 1

Window Size: 0

Checksum: 0x0000

Fragmentation

When sending a file, the user can set fragment size or without setting the value, it will be the maximum that the protocol can transfer at one time. Also, when the user selects a value, there is a check whether it exceeds the capabilities of the protocol. The required file will be downloaded and divided into parts of the length that was

previously specified. In the protocol header, in place of total fragments, the number of fragments into which the file was divided will be set and will be sent to another node.

When receiving data with fragmentation, its sequential number and the integrity of the data will be checked, and only after receiving all parts, the fragments will be combined.

Integrity verification

To check the integrity of the data, the program will use CRC-16 (Cyclic Redundancy Check). Before sending the data, the CRC will be calculated and placed in the space allocated for it in the header. When receiving data, the CRC will be checked with the received one to make sure that the data was not damaged during sending.

To generate a CRC, you first need to decide on a polynomial, since the polynomial determines how the checksum will be calculated. The polynomial used to calculate the CRC has a certain length. For CRC-16, this is 16 bits. Then you need to add the length of one bit less to the original message. This is done to account for the length of a possible remainder. These added bits are zeros. Next, a binary division operation is performed. The original message with the added zeros is divided by the polynomial using XOR. After the division is complete, the remainder becomes the CRC value. It is a checksum for the original message.

Example:

1011 $\overline{)} 10101111000$

1011

1111

1011

1001

1011

1000

1011

110

Divider (polynomial): 1011

Remainder (CRC): 110

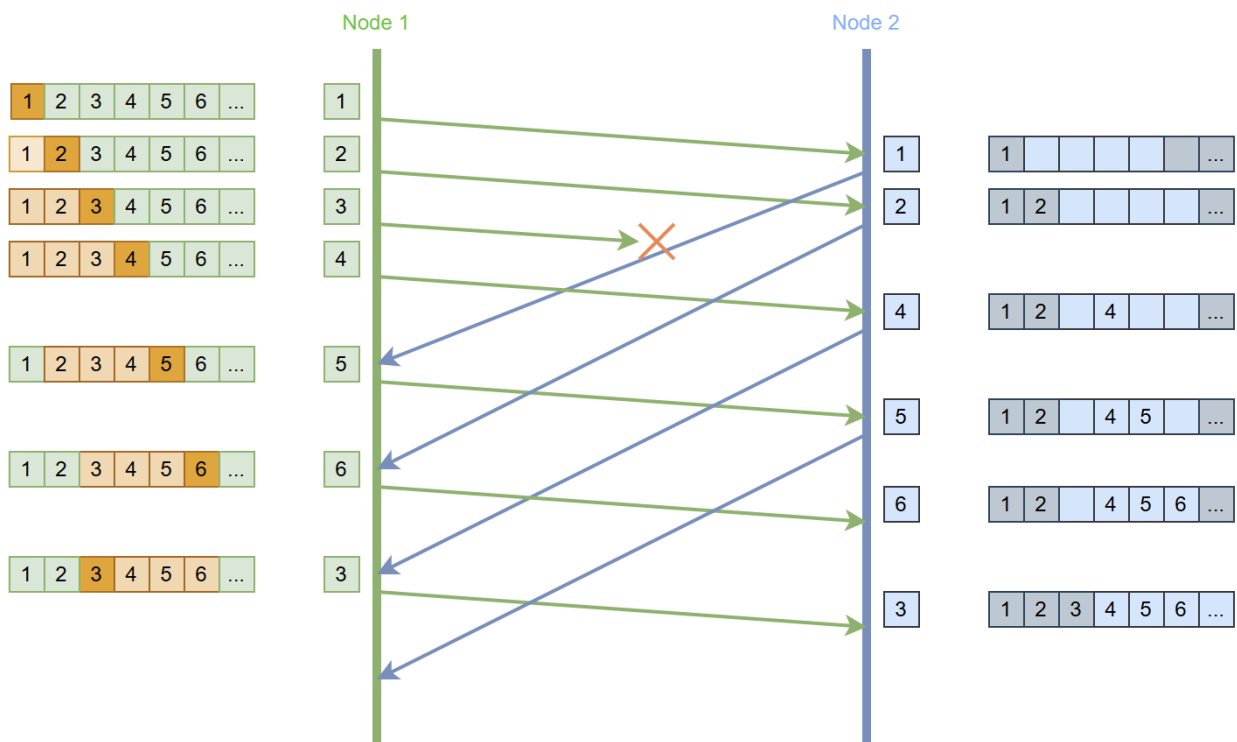
This checksum was chosen because the CRC is calculated faster and requires fewer resources than MD5 or SHA. CRC-16 will be used because CRC-8 is too small to reliably detect errors, potentially missing some types of errors. While CRC-32 is more reliable than CRC-16, it demands more resources and performance than unnecessary for this project.

Automatic Repeat Request (ARQ)

As mentioned earlier in the project, the data is divided into fragments. Each fragment will contain a unique sequence number, and the header of each packet indicates the total number of fragments. The receiver acknowledges each successfully received fragment with a separate ACK message. If a fragment is damaged, the receiver will send NACK, and the sender retransmits only the missing fragment. If a fragment is not received, the receiver does not send an ACK for this fragment, and the sender retransmits only the missing fragment.

The concept of a sliding window is used. The sender's window and the receiver's window allow multiple fragments to be transmitted without waiting for each one to be acknowledged.

For example, as shown in the image, node 1 transmits data to node 2 using the sliding window mechanism. Node 1 sends multiple frames in a row, each of which can be acknowledged independently. Node 2 receives the frames, but frame 3 is lost, so it is not acknowledged. Node 2 sends acknowledgments for all successfully received frames, despite the loss of frame 3. Node 1 resends only the lost frame 3, after which node 2 acknowledges it.



This mechanism allows for more efficient data transmission management, minimizing retransmissions and increasing throughput. Compared to Go-Back-N, Selective Repeat does not burden the network with retransmissions of all fragments. This reduces redundant data transmission and improves efficiency. Therefore, Selective Repeat is more efficient for this task.

After receiving all fragments, the receiver assembles the data into the original file or message and checks it for integrity using CRC-16. If all fragments are received and assembled correctly, the receiver sends an acknowledgment of transmission completion.

1064...	5214.913238	127.0.0.1	127.0.0.1	PKS	65497 Data Packet (65455 bites)
1064...	5214.970224	127.0.0.1	127.0.0.1	PKS	42 ACK
1064...	5215.014194	127.0.0.1	127.0.0.1	PKS	65497 Data Packet (65455 bites)
1064...	5215.064964	127.0.0.1	127.0.0.1	PKS	42 ACK
1064...	5215.065947	127.0.0.1	127.0.0.1	PKS	65497 Data Packet (65455 bites)
1064...	5215.101462	127.0.0.1	127.0.0.1	PKS	42 ACK
1064...	5215.117329	127.0.0.1	127.0.0.1	PKS	65497 Data Packet (65455 bites)
1064...	5215.151541	127.0.0.1	127.0.0.1	PKS	42 ACK
1064...	5215.168804	127.0.0.1	127.0.0.1	PKS	65497 Data Packet (65455 bites)
1064...	5215.209990	127.0.0.1	127.0.0.1	PKS	42 ACK
1064...	5215.220911	127.0.0.1	127.0.0.1	PKS	65497 Data Packet (65455 bites)
1064...	5215.268010	127.0.0.1	127.0.0.1	PKS	42 ACK
1064...	5215.272291	127.0.0.1	127.0.0.1	PKS	57964 Data Packet (57922 bites)
1064...	5215.314364	127.0.0.1	127.0.0.1	PKS	42 ACK

Changes made during the implementation compared to the proposed solution:

The receiver can send NACK in case the fragment is damaged and the sender in this case will resend the needed fragment.

Maintain the connection (Keep-Alive)

There is a special flag in the header that specifies that the message will be exclusively a connection confirmation. These messages will not contain a payload, but only signal that the connection is active.

If no data is being transferred, a timer will initiate a heartbeat message every 5 seconds. If no response is received within 3 consecutive heartbeat messages, this means that the connection is lost. If the connection is lost during data transfer, the system should send a heartbeat to check if the connection has been restored.

If there is no response after several attempts, the connection will be considered lost and the file transfer will be considered unsuccessful. If a response comes, the file transfer continues from the point where it was interrupted, the sender repeats the fragments that were not acknowledged by the receiver.

87354	4783.905905	127.0.0.1	127.0.0.1	PKS	42 Keep-Alive
87355	4783.906077	127.0.0.1	127.0.0.1	PKS	42 ACK Keep-Alive
87358	4785.337319	127.0.0.1	127.0.0.1	PKS	42 Keep-Alive
87359	4785.337488	127.0.0.1	127.0.0.1	PKS	42 ACK Keep-Alive
87601	4788.907191	127.0.0.1	127.0.0.1	PKS	42 Keep-Alive
87602	4788.907366	127.0.0.1	127.0.0.1	PKS	42 ACK Keep-Alive
87605	4790.338534	127.0.0.1	127.0.0.1	PKS	42 Keep-Alive
87606	4790.338706	127.0.0.1	127.0.0.1	PKS	42 ACK Keep-Alive

1071...	5230.517272	127.0.0.1	127.0.0.1	PKS	42 Keep-Alive
1071...	5230.517472	127.0.0.1	127.0.0.1	PKS	42 ACK Keep-Alive

```

Frame 107173: 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface \Device
Null/Loopback
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
User Datagram Protocol, Src Port: 2222, Dst Port: 1111
Custom Protocol: Control Message
  Flags: 0x10
  Sequence Number: 678
  Acknowledgment Number: 977
  Total Fragments: 1
  Window Size: 0
  Checksum: 0x0000

```

1071...	5230.517272	127.0.0.1	127.0.0.1	PKS	42 Keep-Alive
1071...	5230.517472	127.0.0.1	127.0.0.1	PKS	42 ACK Keep-Alive

```

Frame 107174: 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface \Device
Null/Loopback
Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
User Datagram Protocol, Src Port: 1111, Dst Port: 2222
Custom Protocol: Control Message
  Flags: 0x50
  Sequence Number: 977
  Acknowledgment Number: 677
  Total Fragments: 1
  Window Size: 0
  Checksum: 0x0000

```

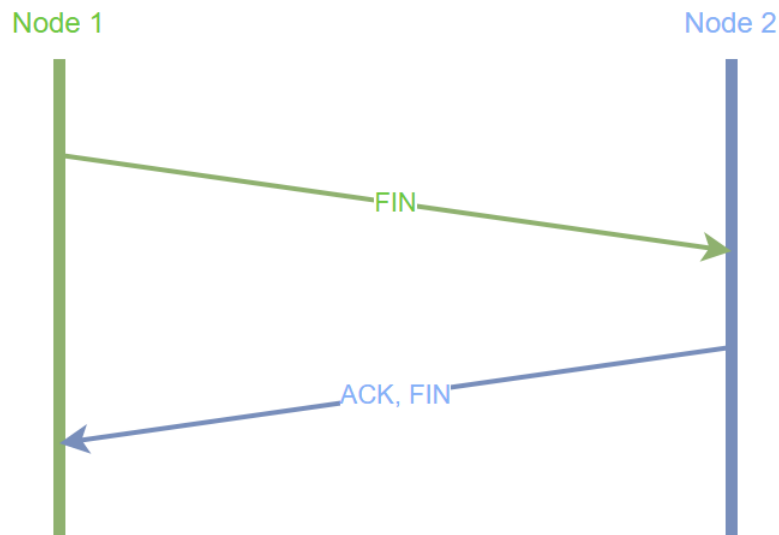
The loss of connection

1374...	5925.756179	127.0.0.1	127.0.0.1	PKS	42 Keep-Alive
1374...	5930.758023	127.0.0.1	127.0.0.1	PKS	42 Keep-Alive
1375...	5935.759274	127.0.0.1	127.0.0.1	PKS	42 Keep-Alive
1375...	5940.760664	127.0.0.1	127.0.0.1	PKS	42 RST

Connection completion

To terminate the connection whenever the user wants it, a 2-way handshake will be used. The 2-way handshake will ensure that the connection is terminated gracefully, allowing both sides to confirm the termination. As shown in the diagram, one node initiates the connection termination by sending a termination flag (FIN). The second node responds with an ACK-FIN, acknowledging that it has received the termination request and closed the connection. The first node receives ACK-FIN and closed the connection too.

The RST flag is used to force termination in the event of errors or a connection failure. If a node decides to terminate the connection immediately, it sends a message with the RST flag. The node that receives the RST immediately terminates the connection without any additional messages.



73865	4420.155896	127.0.0.1	127.0.0.1	PKS	42 FIN
73866	4420.156899	127.0.0.1	127.0.0.1	PKS	42 ACK-FIN

73865	4420.155896	127.0.0.1	127.0.0.1	PKS	42 FIN
73866	4420.156899	127.0.0.1	127.0.0.1	PKS	42 ACK-FIN
▶ Frame 73865: 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface \ ▶ Null/Loopback ▶ Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1 ▶ User Datagram Protocol, Src Port: 1111, Dst Port: 2222 ▼ Custom Protocol: Control Message - Flags: 0x04 - Sequence Number: 402 - Acknowledgment Number: 233 - Total Fragments: 1 - Window Size: 0 - Checksum: 0x0000					

73865	4420.155896	127.0.0.1	127.0.0.1	PKS	42 FIN
73866	4420.156899	127.0.0.1	127.0.0.1	PKS	42 ACK-FIN
▶ Frame 73866: 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface ▶ Null/Loopback ▶ Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1 ▶ User Datagram Protocol, Src Port: 2222, Dst Port: 1111 ▼ Custom Protocol: Control Message - Flags: 0x44 - Sequence Number: 233 - Acknowledgment Number: 402 - Total Fragments: 1 - Window Size: 0 - Checksum: 0x0000					

Changes made during the implementation compared to the proposed solution:

The 4-way handshake was replaced by 2-way handshake because 2-way handshake is enough to end connection with acknowledgment from the other side.

Make a mistake intentionally

The user can select a scenario in which the program makes a mistake when transmitting data. When forming a data packet, the checksum value changes to an incorrect one. This allows testing of how the system handles damaged data.

1230...	5568.315511	127.0.0.1	127.0.0.1	PKS	65497 Data Packet (65455 bites)
1230...	5568.353702	127.0.0.1	127.0.0.1	PKS	42 ACK
1230...	5568.370730	127.0.0.1	127.0.0.1	PKS	65497 Data Packet (65455 bites)
1230...	5568.416082	127.0.0.1	127.0.0.1	PKS	42 ACK
1230...	5568.422055	127.0.0.1	127.0.0.1	PKS	65497 Data Packet (65455 bites)
1230...	5568.456882	127.0.0.1	127.0.0.1	PKS	42 ACK
1230...	5568.476703	127.0.0.1	127.0.0.1	PKS	65497 Data Packet (65455 bites)
1230...	5568.517064	127.0.0.1	127.0.0.1	PKS	42 ACK
1230...	5568.531810	127.0.0.1	127.0.0.1	PKS	65497 Data Packet (65455 bites)
1230...	5568.573927	127.0.0.1	127.0.0.1	PKS	42 ACK
1230...	5568.583394	127.0.0.1	127.0.0.1	PKS	65497 Data Packet (65455 bites)
1230...	5568.623421	127.0.0.1	127.0.0.1	PKS	42 NACK
1230...	5568.638605	127.0.0.1	127.0.0.1	PKS	57964 Data Packet (57922 bites)
1230...	5568.642866	127.0.0.1	127.0.0.1	PKS	65497 Data Packet (65455 bites)
1230...	5568.677458	127.0.0.1	127.0.0.1	PKS	42 ACK
1230...	5568.711051	127.0.0.1	127.0.0.1	PKS	42 ACK

Conclusion

This project demonstrates a custom peer-to-peer (P2P) protocol designed and implemented over the User Datagram Protocol (UDP). The protocol facilitates communication between two nodes in an Ethernet network, enabling both text and file exchanges while ensuring reliable data transmission.

The protocol uses a three-way handshake for connection establishment, a keep-alive mechanism to maintain continuous connectivity and a two-way handshake for graceful termination. It also includes a reset (RST) for forced connection termination in case of failures.

The application supports dynamic fragmentation of files to ensure reliable transfer of large data, with sequence numbers and CRC-16 checks for error detection. This guarantees that all data fragments are received and assembled correctly before delivery to the receiver.

Selective Repeat ARQ is implemented in the protocol, allowing for efficient retransmission of lost or corrupted fragments without redundant retransmissions of successfully received data, thus optimizing throughput and reducing network load.

Sources:

How to calculate the CRC Date accessed: October 20, 2024

<https://www.educative.io/answers/how-to-calculate-the-crc>

Difference Between Stop and Wait, GoBackN and Selective Repeat

Date accessed: October 20, 2024

<https://www.geeksforgeeks.org/difference-between-stop-and-wait-gobackn-and-selective-repeat/>

Sliding Window Protocol: Set 3 (Selective Repeat) Date accessed: October 20, 2024

<https://www.geeksforgeeks.org/sliding-window-protocol-set-3-selective-repeat/>

Understanding TCP Flags Date accessed: October 20, 2024

<https://www.site24x7.com/learn/linux/tcp-flags.html>

What is TCP (Transmission Control Protocol)? Date accessed: October 20, 2024

<https://www.geeksforgeeks.org/what-is-transmission-control-protocol-tcp/>